Arrays are considered to be equal if they contain the same key/value pairs. Types are coerced for both keys and values: we can see this when comparing the four indexed arrays (`$indexed4` is technically an associative array because not all of its keys are integers). When using the weakly-typed == operator, all four are considered equal. Ditto for the associative `$assoc1` and `$assoc2`.

However, using the strict === operator, type *and* element order are also taken into account. `$indexed1 === $indexed3` evaluates to `false` because types do not match, `$indexed1 === $indexed4` is false because of types and order, `$assoc1 === $assoc2` is `false` because the order of elements is different.

Finding Uncommon Elements

PHP provides a built-in function, `array_diff()`, which will compare an array to one or more comparison arrays and return all values that are not found. Keys are not compared nor is order important. If an element occurs multiple times in the first array it only needs to match a single time with the second (or subsequent) arrays to be discarded from the returned set. However, the inverse is not true: all occurrences of an unmatched element will be included in the return:

```
$car1 = ['mark' => 1, 'Golf', 'VW', 'Golf', 'GTI', 'GTI'];
$car2 = ['GTI', 'VW', 'Scirocco', '1'];

print_r(array_diff($car1, $car2));

Output:
Array
(
    [0] => Golf
    [2] => Golf
)
```

`$car1` and `$car2` both contain the strings `'VW'` and `'GTI'`, but `'Golf'` is not found in `$car2`. This is therefore considered a difference and is added to the returned array twice because it is found twice in `$car1`. Type coercion is in force here and `'mark' => 1` is considered equal to `'1'`. Finally, `'GTI'` occurs twice in `$car1` but, because it is also found at least once in `$car2`, it is not considered a difference, and it is omitted from